



Universidade Estácio - Polo EAD Barbacena-MG

Curso: Desenvolvimento Full-Stack

Disciplina: DGT2811 Desenvolvimento Back-End Corporativo com Java e Cloud

Turma: 2026.3

Semestre Letivo: Último

Nome do Aluno: Daniel de Paula Magalhães

Matrícula: 202307322008

Link do Repositório:

[DaniBoy-code/DGT2811-Desenvolvimento-Back-End-Corporativo-com-Java-E-Cloud](https://github.com/DaniBoy-code/DGT2811-Desenvolvimento-Back-End-Corporativo-com-Java-E-Cloud)

Integração de sistemas cadastrais com interface Web(Servlets, JPA, JEE, e Bootstrap).

Objetivo: Implementar a persistência de dados com JPA, criar regras de negócio utilizando EJBs na plataforma JEE, e desenvolver um sistema cadastral Web baseado na arquitetura MVC utilizando Servlets como controladores (front), JSPs como visualizações e o framework Bootstrap para design responsivo.

Códigos solicitados:

Entidade JPA (Produto.java):

```
package cadastroee.model;

import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import jakarta.xml.bind.annotation.XmlRootElement;
import java.io.Serializable;

/**
 *
 * @author daniel
 */
@Entity
@Table(name = "Produto")
@XmlRootElement
@NamedQueries({
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p"),
    @NamedQuery(name = "Produto.findById", query = "SELECT p FROM Produto p WHERE p.id = :id"),
    @NamedQuery(name = "Produto.findByName", query = "SELECT p FROM Produto p WHERE p.nome = :nome"),
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM Produto p WHERE p.quantidade = :quantidade"),
    @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p FROM Produto p WHERE p.precoVenda = :precoVenda")
})
public class Produto implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
```

```

@Column(name = "id")
private Integer id;
@Basic(optional = false)
@NotNull
@Size(min = 1, max = 100)
@Column(name = "nome")
private String nome;
@Basic(optional = false)
@NotNull
@Column(name = "quantidade")
private int quantidade;
@Basic(optional = false)
@NotNull
@Column(name = "precoVenda")
private Float precoVenda;
public Produto() {
}
public Produto(Integer id) {
    this.id = id;
}
public Produto(Integer id, String nome, int quantidade, Float precoVenda) {
    this.id = id;
    this.nome = nome;
    this.quantidade = quantidade;
    this.precoVenda = precoVenda;
}
public Integer getId() {
    return id;
}
public void setId(Integer id) {
    this.id = id;
}
public String getNome() {
    return nome;
}
public void setNome(String nome) {
    this.nome = nome;
}
public int getQuantidade() {
    return quantidade;
}
public void setQuantidade(int quantidade) {
    this.quantidade = quantidade;
}
public Float getPrecoVenda() {
    return precoVenda;
}
public void setPrecoVenda(Float precoVenda) {
    this.precoVenda = precoVenda;
}
@Override
public int hashCode() {
    int hash = 0;
    hash += (id != null ? id.hashCode() : 0);
    return hash;
}
@Override
public boolean equals(Object object) {
    if (!(object instanceof Produto)) {
        return false;
    }
    Produto other = (Produto) object;
    if ((this.id == null && other.id != null) || (this.id != null && !this.id.equals(other.id))) {
        return false;
    }
    return true;
}
@Override
public String toString() {
    return "cadastroee.model.Produto[ id=" + id + " ]";
}
}

```

Configuração de Persistência (persistence.xml)

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0"
xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
java.sun.com/xml/ns/persistence http://java.sun.com/xml/ns/persistence/
persistence_1_0.xsd">
  <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
  </persistence-unit>
</persistence>

```

```

        <properties/>
    </persistence-unit>
</persistence>

```

Session Beans (ProdutoFacadeLocal.java e ProdutoFacade.java):

ProdutoFacadeLocal.java:

```

package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Local;
import java.util.List;

@Local
public interface ProdutoFacadeLocal {
    void create(Produto produto);
    void edit(Produto produto);
    void remove(Produto produto);
    Produto find(Object id);
    List<Produto> findAll();
}

```

ProdutoFacade.java:

```

package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Local;
import java.util.List;

@Local
public interface ProdutoFacadeLocal {
    void create(Produto produto);
    void edit(Produto produto);
    void remove(Produto produto);
    Produto find(Object id);
    List<Produto> findAll();
}

```

Controlador Web (ServletProdutoFC.java):

```

package cadastroee.servlets;
import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.List;
@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {
    @EJB
    ProdutoFacadeLocal facade;
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String acao = request.getParameter("acao");
        if (acao == null) {
            acao = "listar";
        }
        String destino = "ProdutoLista.jsp";
        try {
            if (acao.equals("listar")) {
                List<Produto> produtos = facade.findAll();
                request.setAttribute("produtos", produtos);
            } else if (acao.equals("formAlterar")) {
                Integer id = Integer.parseInt(request.getParameter("id"));
                Produto p = facade.find(id);
                request.setAttribute("produto", p);
                destino = "ProdutoDados.jsp";
            } else if (acao.equals("formIncluir")) {
                destino = "ProdutoDados.jsp";
            } else if (acao.equals("excluir")) {
                Integer id = Integer.parseInt(request.getParameter("id"));
                Produto p = facade.find(id);
                facade.remove(p);
                request.setAttribute("produtos", facade.findAll());
            } else if (acao.equals("alterar")) {
                Integer id = Integer.parseInt(request.getParameter("id"));
            }
        }
    }
}

```

```

        String nome = request.getParameter("nome");
        int quantidade = Integer.parseInt(request.getParameter("quantidade"));
        float preco = Float.parseFloat(request.getParameter("precoVenda"));
        Produto p = facade.find(id);
        p.setNome(nome);
        p.setQuantidade(quantidade);
        p.setPrecoVenda(preco);
        facade.edit(p);
        request.setAttribute("produtos", facade.findAll());
    } else if (acao.equals("incluir")) {
        String nome = request.getParameter("nome");
        int quantidade = Integer.parseInt(request.getParameter("quantidade"));
        float preco = Float.parseFloat(request.getParameter("precoVenda"));
        Produto p = new Produto();
        p.setNome(nome);
        p.setQuantidade(quantidade);
        p.setPrecoVenda(preco);
        facade.create(p);
        request.setAttribute("produtos", facade.findAll());
    }
} catch (Exception e) {
    System.out.println("Erro ao processar acao: " + e.getMessage());
}
request.getRequestDispatcher(destino).forward(request, response);
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}
}

```

Páginas JSP (ProdutoLista.jsp e ProdutoDados.jsp):

ProdutoLista.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Lista de Produtos</title>
<!-- Inclusão do Bootstrap pedida no 3º Procedimento -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body class="container">
<h1 class="mt-4">Lista de Produtos</h1>

<a href="ServletProdutoFC?acao=formIncluir" class="btn btn-primary m-2">Novo Produto</a>

<table class="table table-striped">
<thead class="table-dark">
<tr>
<th>ID</th>
<th>Nome</th>
<th>Quantidade</th>
<th>Preço</th>
<th>Ações</th>
</tr>
</thead>
<tbody>
<%
List<Produto> produtos = (List<Produto>) request.getAttribute("produtos");
if (produtos != null) {
    for (Produto p : produtos) {
%>
<tr>
<td><%= p.getId() %></td>
<td><%= p.getNome() %></td>
<td><%= p.getQuantidade() %></td>
<td><%= p.getPrecoVenda() %></td>
<td>
<a href="ServletProdutoFC?acao=formAlterar&id=<%= p.getId() %>" class="btn btn-primary btn-sm">Alterar</a>
<a href="ServletProdutoFC?acao=excluir&id=<%= p.getId() %>" class="btn btn-danger btn-sm">Excluir</a>

```

```

        </td>
      </tr>
    <%
      }
    }
  %>
</tbody>
</table>
</body>
</html>

```

ProdutoDados.jsp:

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Dados do Produto</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
  </head>
  <body class="container">
    <%
      Produto produto = (Produto) request.getAttribute("produto");
      String acao = (produto == null) ? "incluir" : "alterar";
      String titulo = (produto == null) ? "Incluir Produto" : "Alterar Produto";
    %>
    <h1 class="mt-4"><%= titulo %></h1>
    <form action="ServletProdutoFC" method="POST" class="form">
      <input type="hidden" name="acao" value="<%= acao %>">
      <% if (produto != null) { %>
        <input type="hidden" name="id" value="<%= produto.getId() %>">
      <% } %>
      <div class="mb-3">
        <label for="nome" class="form-label">Nome:</label>
        <input type="text" class="form-control" id="nome" name="nome"
          value="<%= produto != null ? produto.getNome() : "" %>" required>
      </div>
      <div class="mb-3">
        <label for="quantidade" class="form-label">Quantidade:</label>
        <input type="number" class="form-control" id="quantidade" name="quantidade"
          value="<%= produto != null ? produto.getQuantidade() : "" %>" required>
      </div>
      <div class="mb-3">
        <label for="precoVenda" class="form-label">Preço de Venda:</label>
        <input type="number" step="0.01" class="form-control" id="precoVenda" name="precoVenda"
          value="<%= produto != null ? produto.getPrecoVenda() : "" %>" required>
      </div>
      <button type="submit" class="btn btn-primary">Salvar</button>
      <a href="ServletProdutoFC?acao=listar" class="btn btn-secondary">Cancelar</a>
    </form>
  </body>
</html>

```

RESULTADOS DE EXECUÇÃO

Teste Inicial:

← ↻ 🏠 📄 localhost:8080/CadastroEE-war/ProdutoServlet

🔍 Importar favoritos 📁 i5 12600kf

Lista de Produtos

- Teclado Mecânico - 250.0
- Mouse Gamer - 120.5
- Monitor 24 - 850.0
- Teclado Mecânico - 250.0
- Mouse Gamer - 120.5
- Monitor 24 - 850.0
- dada - 2500.0
- mouse gamer - 300.0
- RTX 4090 - 3500.0
- Teclado Mecânico - 250.0
- Mouse Sem Fio - 85.5
- Monitor 24 polegadas - 800.0
- Fone Bluetooth - 249.0

Lista de Produtos com Bootstrap:

← ↻ 🏠 📄 localhost:8080/CadastroEE-war/ServletProdutoFC

🔍 Importar favoritos 📁 i5 12600kf

Lista de Produtos

Novo Produto

ID	Nome	Quantidade	Preço	Ações
1	Teclado Mecânico	15	250.0	Alterar Excluir
2	Mouse Gamer	30	120.5	Alterar Excluir
3	Monitor 24	10	850.0	Alterar Excluir
4	Teclado Mecânico	15	250.0	Alterar Excluir
5	Mouse Gamer	30	120.5	Alterar Excluir
6	Monitor 24	10	850.0	Alterar Excluir
7	dada	5	2500.0	Alterar Excluir
8	mouse gamer	5	300.0	Alterar Excluir
9	RTX 4090	1	3500.0	Alterar Excluir
11	Teclado Mecânico	15	250.0	Alterar Excluir
12	Mouse Sem Fio	30	85.5	Alterar Excluir
13	Monitor 24 polegadas	10	800.0	Alterar Excluir
14	Fone Bluetooth	13	249.0	Alterar Excluir

Formulário de Inclusão de itens:

← ↻ 🏠 📄 localhost:8080/CadastroEE-war/ServletProdutoFC?acao=formIncluir

🔍 Importar favoritos 📁 i5 12600kf

Incluir Produto

Nome:

cpu

Quantidade:

12

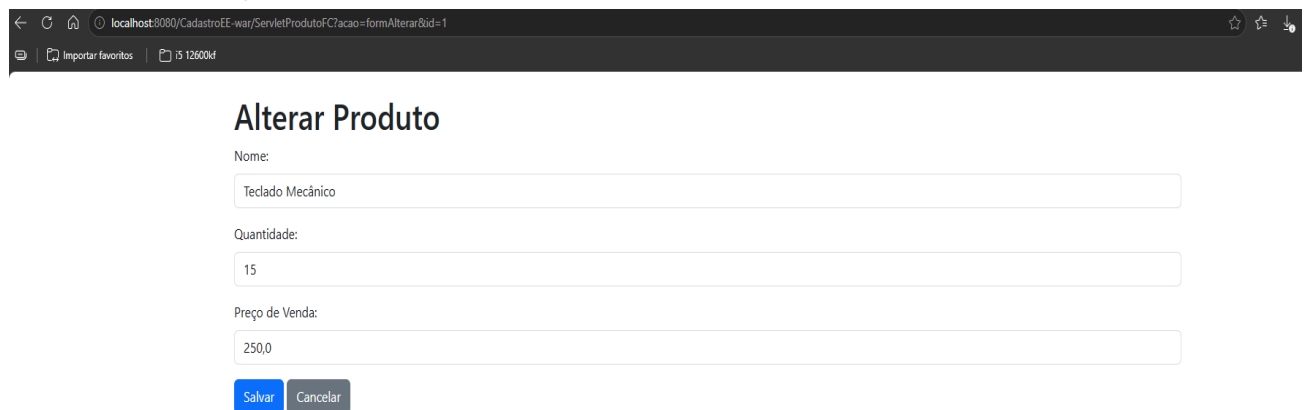
Preço de Venda:

2539

Salvar

Cancelar

Formulário de Alteração:



localhost:8080/CadastroEE-war/ServletProdutoFC?acao=formAlterar&id=1

Importar favoritos 15.12600ld

Alterar Produto

Nome:

Quantidade:

Preço de Venda:

Salvar Cancelar

Respostas da questões solicitadas:

Como é organizado um projeto corporativo no NetBeans? Um projeto corporativo (Enterprise Application) no NetBeans é organizado de forma modular para separar responsabilidades. É criado um projeto principal (EAR) que encapsula módulos dependentes: o módulo EJB (para a lógica de negócio e persistência de dados) e o módulo Web/WAR (para a interface do utilizador, contendo Servlets e JSPs).

Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java? A JPA (Java Persistence API) é responsável pelo mapeamento objeto-relacional (ORM), convertendo as tabelas da base de dados relacional em classes Java de forma transparente. Os EJBs (Enterprise JavaBeans) atuam como a camada de serviço e negócio, gerindo a comunicação com a base de dados através do Entity engineering e assegurando o controle automático de transações.

Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB? O NetBeans oferece assistentes de geração de código (wizards) que automatizam tarefas morosas. Permite gerar classes de Entidade (JPA) diretamente a partir de tabelas existentes na base de dados e criar automaticamente os Session Beans (EJBs de Facade) para cada entidade, gerando os métodos CRUD básicos de forma instantânea.

O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes? Servlets são classes Java que correm no servidor web e intercetam pedidos HTTP (GET, POST), processam a lógica necessária e devolvem uma resposta ao cliente. O NetBeans suporta a sua criação através de assistentes que geram a estrutura básica do ficheiro, incluindo os métodos doGet e doPost, e configuram as anotações web modernas (@WebServlet).

Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs? A comunicação é feita através de Injeção de Dependências. No Servlet, declara-se um atributo com a interface do EJB (ex: ProdutoFacadeLocal) e anota-se com @EJB. O contentor (GlassFish) encarrega-se de injetar a instância correta do bean no momento da execução, permitindo ao Servlet chamar os métodos da base de dados.

Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC? O padrão Front Controller centraliza todos os pedidos numa única entrada. Na arquitetura MVC, o ServletProdutoFC atua como o *Controller*. Ele recebe todos os pedidos, analisa um parâmetro específico (como acao), executa a lógica apropriada invocando o EJB (*Model*), e redireciona os resultados para a página JSP correspondente (*View*).

Quais as diferenças e semelhanças entre Servlets e JSPs? Ambos processam pedidos web dinâmicos e, na verdade, as JSPs são compiladas em Servlets pelo servidor. A diferença principal é que os Servlets são código Java onde se injeta HTML (ideais para o controlo e lógica), enquanto as JSPs são ficheiros baseados em HTML onde se injeta código Java (ideais para a camada de visualização).

Qual a diferença entre um redirecionamento simples e o uso do método forward? Para que servem parâmetros e atributos? O redirecionamento simples (`sendRedirect`) pede ao navegador para fazer um novo pedido para outro URL, perdendo os dados do pedido original. O `forward` (via `RequestDispatcher`) ocorre internamente no servidor, mantendo o URL inalterado e preservando os dados enviados. Os parâmetros são dados enviados pelo cliente (navegador, formulários ou URL), enquanto os atributos são objetos partilhados pelo servidor para passar dados entre o Servlet e a JSP (como a lista de produtos).

Como o framework Bootstrap é utilizado e por que garante a independência estrutural? O Bootstrap é incluído nas JSPs através de links de CDN (CSS e JavaScript) na secção `<head>`. Ele garante independência estrutural porque separa o conteúdo (HTML) do estilo (CSS). Em vez de escrever regras de estilo personalizadas complexas, o programador aplica classes predefinidas do Bootstrap (ex: `table`, `btn-primary`) diretamente nas tags HTML.

Qual a relação entre o Bootstrap e a responsividade da página? O Bootstrap foi construído sob o princípio *Mobile-First*. Utiliza um sistema de grelhas (*grid system*) e *media queries* no seu CSS que detetam o tamanho do ecrã do dispositivo. Isto faz com que os elementos se reorganizem e redimensionem automaticamente, garantindo que o sistema seja perfeitamente utilizável em telemóveis, tablets e computadores.